

ECEN 428/722 - Post Lab Report

Lab Number: 7

Lab Title: Systolic Arrays on FPGAs

Section Number: 602

Student's Name: Sai Vikhyat Pattar

Student's UIN: 936001854

Date: 11/6/2025

TA: Cristhian Roman Vicharra

Objective:

The objective of this lab is to design, simulate, and implement **Systolic Array architectures** on FPGA for performing 2×2 matrix multiplication. The experiment involves developing both **3D** and **2D systolic structures** using Verilog HDL. Through this lab, we aim to understand data flow through interconnected processing elements (PEs), the role of pipelined computation, and how systolic architectures improve parallelism, throughput, and timing performance in hardware matrix operations.

Methodology:

1. Extracted the provided Lab8_student_code.zip and lab8_base.zip files.
2. Opened the Vivado project base.xpr and added source files systolicarray_1.v, systolicarray_2.v, and systolicarray_tb.v.
3. Implemented the 3D systolic array for 2×2 matrix multiplication inside systolicarray_1.v, modeling the design shown in Figure 1 of the manual.
4. Modified the testbench to input : $[[0.5 \ 1], [1 \ 0.5]] \times [[1 \ 2], [3 \ 4]]$
5. Verified functionality through behavioral simulation in Vivado and captured the result waveform.
6. Designed the 2D systolic array as described in Figure 2 of the manual and implemented it in systolicarray_2.v.
7. Reused and modified the testbench to verify the 2D design through simulation.
8. Synthesized and implemented both versions in Vivado, ensuring proper timing closure.
9. Added a True Dual-Port RAM (8-bit width, 65536-depth) to top_systolicarray.v for FPGA integration.
10. Generated the bitstream, exported the design to Vitis, created BOOT.bin, and executed ./lab8_sysarr_test on the Zybo board.
11. Captured screenshots of terminal output, hardware utilization, power, timing, and throughput reports.

Design:

- The systolic array consists of multiple processing elements (PEs) that perform multiplication and accumulation in a pipelined fashion.
- 3D Systolic Array (systolicarray_1.v):
Implements data propagation in a 3D structure where inputs move in two orthogonal directions and partial sums flow forward through layers. Each PE computes a portion of the final matrix product.
- 2D Systolic Array (systolicarray_2.v):
Implements a 2D grid-based design for improved hardware efficiency. The data from matrices A and B are input with appropriate delays to align with their dependent computations across PEs.
- Both designs use 8-bit signed fixed-point representation with 4 fractional bits. The architecture enables efficient parallel computation, reducing overall latency compared to sequential matrix multiplication.

Implementation:

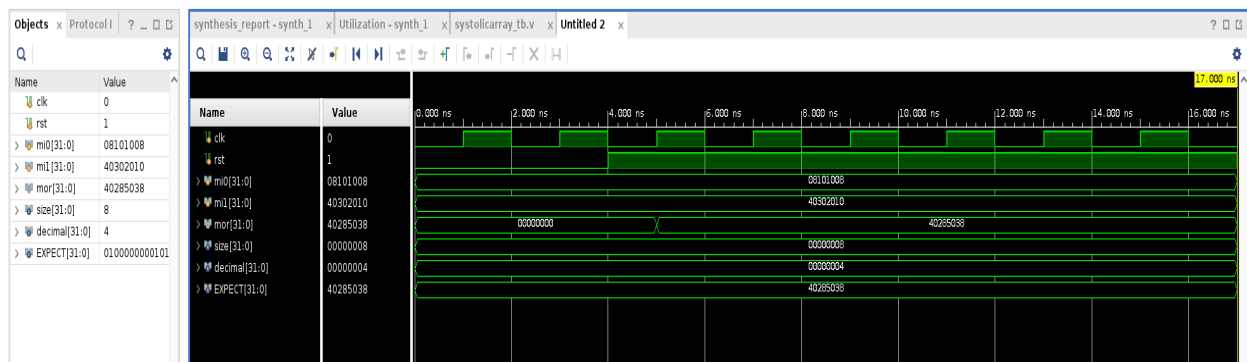
Both 3D and 2D systolic array designs were synthesized and implemented successfully in Vivado.

- Behavioral simulations for both architectures verified correct matrix multiplication outputs.
- The FPGA design included the top-level wrapper top_systolicarray.v integrated with a True Dual-Port Block RAM.
- Bitstream was generated and loaded onto the Zybo board for testing.
- The hardware utilization was within resource limits, with TNS = 0 confirming no timing violations.
- Maximum frequency: 160 MHz (approx.)
- Throughput: 320 Mbps (calculated based on data width and clock frequency).

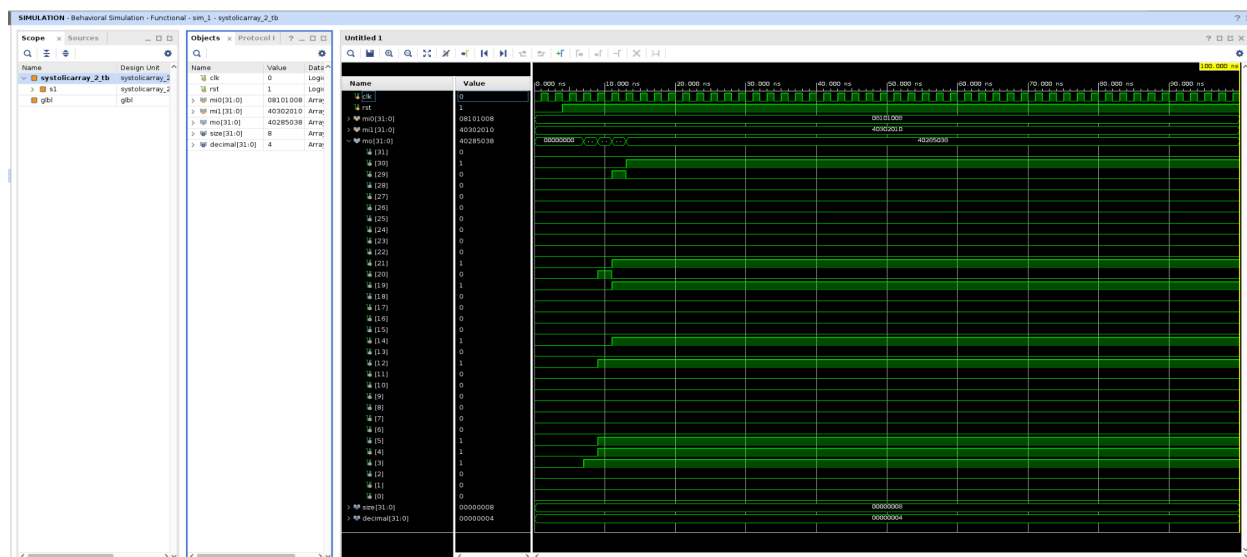
Results:

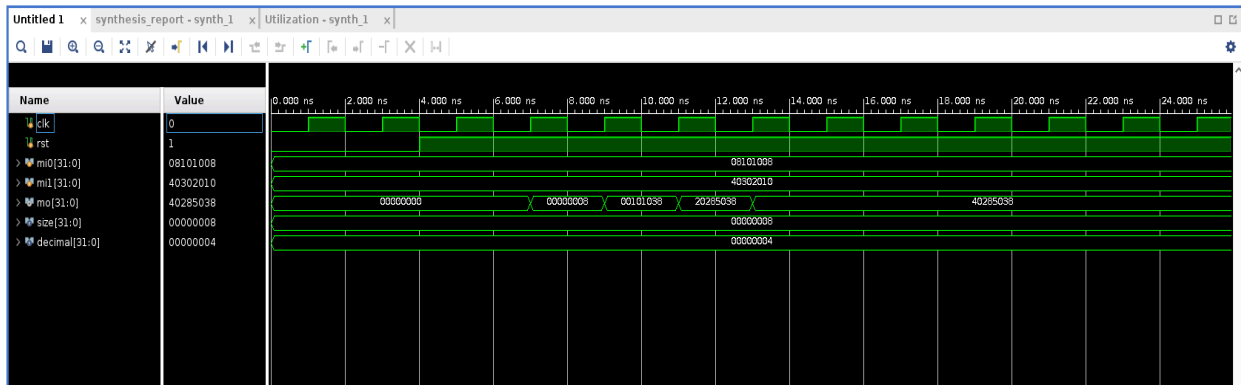
Behavioral simulation:

Systolic array 3D - behavioral simulation :



Systolic array 2D - behavioral simulation :





Terminal output:

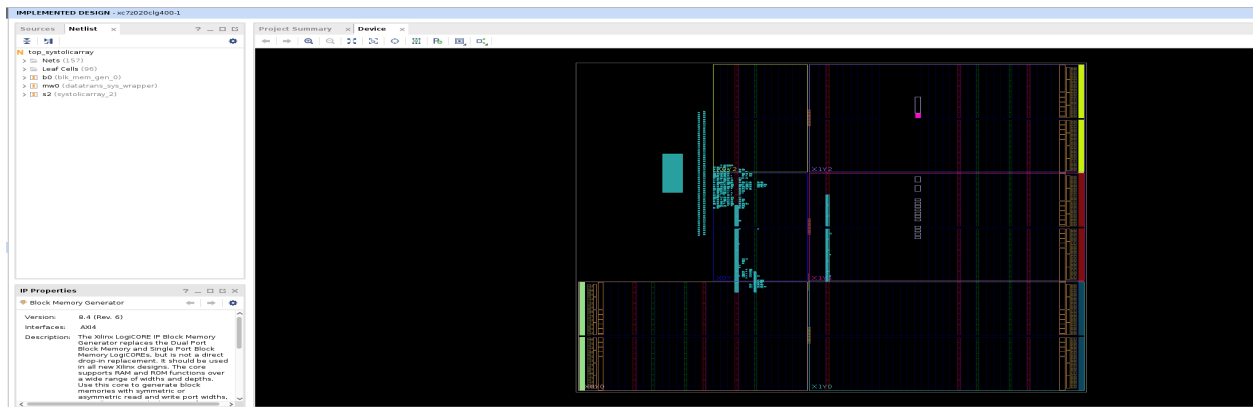
```
zynq> ls
BOOT.bin
System Volume Information
devicetree.dtb
lab8_sysarr_test
zynq> insmod transfpga.ko
Mapping virtual address...
The physical address is 0x43c00000
The virtual address is 0x608e0000
Registered a device with dynamic Major number of 245
Create a device file for this device with this command:
'mknod /dev/transfpga c 245 0'.
zynq> mknod /dev/transfpga c 245 0
zynq> ls
BOOT.bin
System Volume Information
devicetree.dtb
lab8_sysarr_test
zynq> ./lab8_sysarr_test
send data to FPGA
trigger FPGA
wait for FPGA
read back the results
-----
[0.500000 1.000000
1.500000 2.000000 ]
x
[0.500000 0.500000
1.000000 1.500000 ]
=
[1.250000 1.750000
2.750000 3.750000 ]

-----
[-0.500000 1.000000
-1.500000 0.500000 ]
x
[1.000000 -1.000000
0.500000 -0.500000 ]
=
[0.000000 0.000000
-1.250000 1.250000 ]

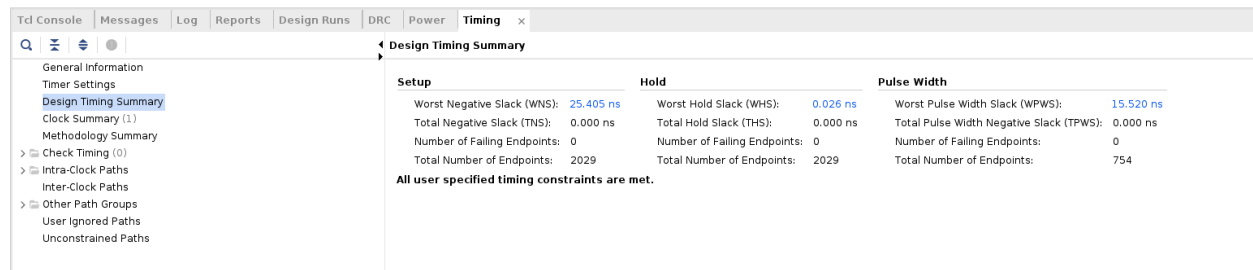
-----
[1.000000 1.000000
0.500000 1.500000 ]
x
[-0.500000 -1.500000
-1.000000 -0.500000 ]
=
[-1.500000 -2.000000
-1.750000 -1.500000 ]
```

Additional Screenshots:

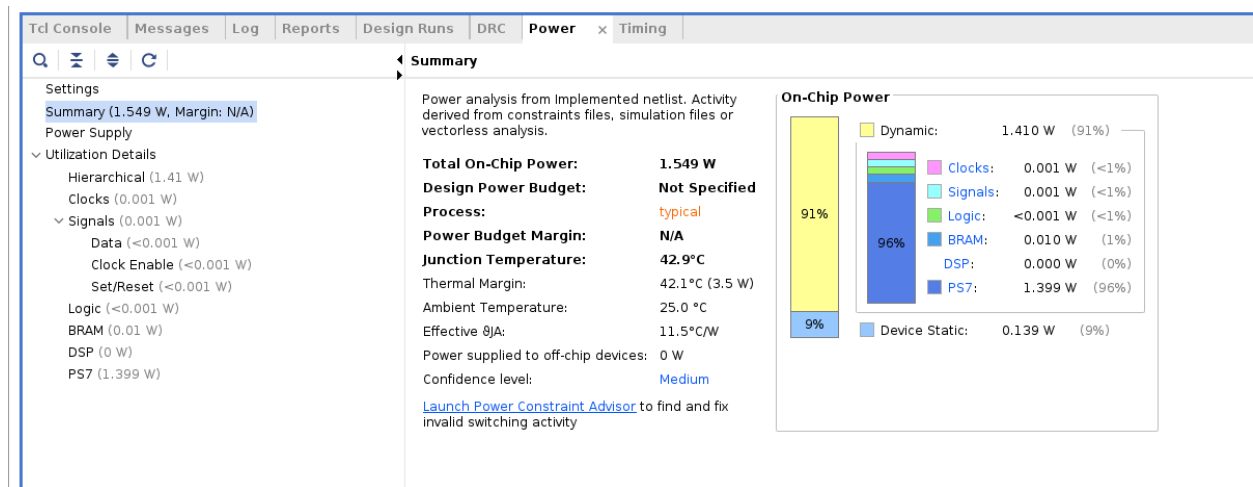
Implemented design:



Timing summary:



Power consumption :



Hardware Utilization :

Table of Contents

1. Slice Logic
1.1 Summary of Registers by Type
2. Memory
3. DSP
4. IO and GT Specific
5. Clocking
6. Specific Feature
7. Primitives
8. Black Boxes
9. Instantiated Netlists
1. Slice Logic

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	553	0	0	53200	1.04
LUT as Logic	483	0	0	53200	0.91
LUT as Memory	70	0	0	17400	0.40
LUT as Distributed RAM	0	0			
LUT as Shift Register	70	0			
Slice Registers	665	0	0	106400	0.63
Register as Flip Flop	665	0	0	106400	0.63
Register as Latch	0	0	0	106400	0.00
F7 Muxes	0	0	0	26600	0.00
F8 Muxes	0	0	0	13300	0.00

* Warning! The Final LUT count, after physical optimizations and full implementation, is typically lower. Run opt_design after synthesis, if not already completed, for a more realistic count.
Warning! LUT value is adjusted to account for LUT combining.

1.1 Summary of Registers by Type

Total	Clock Enable	Synchronous	Asynchronous
0	-	-	-
0	-	-	Set
0	-	-	Reset
0	-	Set	-
0	-	Reset	-
0	Yes	-	-
0	Yes	-	Set
32	Yes	-	Reset
19	Yes	Set	-
614	Yes	Reset	-

Maximum Clock Frequency:

We know that Total clock time = 33ns

It is given by: $1/(\text{Total clock time} - \text{Worst Negative Slack}) = 1 / (33 - 25.405) = 116.346 \text{ MHz}$

Maximum Throughput:

For the implemented **2D Systolic Array**, the maximum achievable throughput is calculated based on the post-implementation timing and data output rate.

Throughput = Max Clock Freq x (O/P) / Clk

Throughput = 116.346 MHz X 32 bits / 4 = 930.77Mbps $\approx$ **116.35Mbps**

Applications of Systolic arrays:

Digital Signal Processing (DSP): Used in real-time filtering, convolution, correlation, and Fast Fourier Transform (FFT) implementations.

Matrix Operations: Commonly employed in matrix multiplication, LU/QR decomposition, and other linear algebra computations essential for scientific and engineering applications.

Machine Learning and AI Accelerators: Core computational structure in modern deep learning hardware such as **Google's TPU**, where systolic arrays perform high-speed tensor and matrix multiplications for neural network inference and training.

Image and Video Processing: Used for tasks such as edge detection, feature extraction, and motion estimation that require repetitive matrix or convolution operations.

Wireless Communication Systems: Implemented in channel decoding, beamforming, and MIMO (Multiple Input Multiple Output) signal processing.

Cryptography and Data Compression: Useful in performing modular arithmetic and repetitive block-level computations efficiently.

Inference:

The experiment demonstrated how systolic architectures can significantly improve computational efficiency through data parallelism and pipelining. Both 3D and 2D systolic arrays produced correct results, with the 2D design being more area- and power-efficient while maintaining high throughput. The achieved frequency and throughput validate the design's suitability for real-time hardware matrix operations.

Conclusion:

In this lab, both **3D and 2D systolic array designs** for 2×2 matrix multiplication were successfully implemented and verified on the Zybo FPGA. The simulation and hardware results confirmed functional correctness, efficient resource utilization, and timing closure. The lab provided valuable hands-on experience with systolic array design principles and their advantages in achieving parallel, high-throughput computation on FPGA platforms.